

The Price of Conservatism: Empirical Evidence from Cloud Right-Sizing and Reservation Heuristics

Ibrahim Mohamed, Mariam Emad, Hazem Ibrahim, Ahmed Sameh, Mahmoud Ahmed, John Ehab

Academic supervision and project support: Dr. Hafez Seliem, Mohamed Essam, and Saif AlDeen Sameh.

All authors: Department of Computer Systems
Faculty of Computer Science
Ain Shams University
Cairo, Egypt

Abstract—Reported production right-sizing systems size resources from high utilization percentiles with headroom, while capacity-reservation theory prescribes price-aware commitment levels that may be much lower under two-tier reserved/on-demand pricing. This paper operationalizes that tension. We use Smart Cloud Optimizer, an implemented AWS cost-optimization pipeline, as a reproducible vehicle for evaluating when conservative $q_{0.95} \times 1.3$ right-sizing heuristics and reservation policies diverge under intermittent demand surges. The system context includes telemetry collection, forecasting, mixed-integer right-sizing, reserved-pricing checks, and service-level waste rules; the research contribution is the empirical bridge between this FinOps-style practice and capacity-reservation theory. On the repository’s synthetic AWS account, a pricing audit finds that several demo EC2 candidate prices are inconsistent with the verified AWS snapshot; the paper therefore reports synthetic right-sizing on a corrected shared EC2 price basis. Under that basis, the deployed $q_{0.95} \times 1.3$ compute MILP is feasible, but the service split is mixed: EC2 cost increases by 15.0% while demo-priced RDS cost decreases by 91.2%, yielding an aggregate compute-scope decrease of 19.0%. A labeled usage-hour reservation replay surrogate recommends 6 EC2 commitments saving \$209.15/mo on eligible EC2 on-demand spend; its 7-, 30-, and 60-day windows collapse to the same synthetic result. On restored raw Bitbrains and Materna GWA traces priced against a snapshot-dated AWS catalog, the right-sizing MILP saves 41.8% in aggregate (bootstrap 95% CI 35.9–47.5%) relative to a like-for-like lift-and-shift baseline, rising to 63.4% with one-year reserved pricing. The Azure Public Dataset V2 arm remains a previously generated boundary case because its raw `vmtable.csv.gz` was not restored: max-based CPU percentiles and unobserved memory can make right-sizing alone cost-negative. Finally, out-of-sample policy simulation quantifies the “price of conservatism” with newsvendor, SAA, dual-capacity, and look-back replay policies: at $\gamma = 0.63$, using the high-percentile capacity rule as a reservation-sizing rule realizes 101.8% and 153.5% of on-demand cost on the synthetic and real traces, whereas the newsvendor fractile realizes 73.6% and 86.2%, and the real-trace 30-day look-back replay realizes 84.9%. We do not claim superiority over AWS, Azure, or commercial FinOps recommenders; no direct evidence for that comparison exists in the repository.

Index Terms—cloud cost optimization, capacity reservation, right-sizing, reserved instances, newsvendor, FinOps, time-series forecasting.

I. INTRODUCTION

Public-cloud users face a recurring capacity trade-off. On-demand capacity is elastic but expensive per unit; reserved or committed capacity is cheaper but can be wasted when

demand falls. The trade-off is most visible under intermittent demand surges, where a workload has a stable base load punctuated by spikes. In that regime, the engineering instinct to provision against high percentiles and the economic incentive to reserve only the cost-effective base load can point in different directions.

Operations-management work formalizes this problem as capacity reservation under intermittent random demand surges. Chen, Lei, and Moinsadeh [1] show that optimal reserved capacity follows a newsvendor-type critical-fractile rule and that supplementary contracts can be managed by a threshold policy. In reported production engineering practice, however, capacity limits and right-sizing recommendations are based on high usage percentiles with headroom, as in the percentile-based approaches reported for large-scale production systems [2]–[4]. Those rules are defensible for performance-risk management, but they are not automatically cost-optimal for reservation sizing.

This paper turns Smart Cloud Optimizer into a research artifact around that distinction. The repository implements telemetry ingestion, forecasting, PuLP-based right-sizing, and transparent pricing/waste rules; dashboard, API, storage, and LLM-advisor components are system context only. The evaluated object is the interaction between practical P95-based right-sizing and price-aware reservation sizing, measured with paper-local scripts on public traces and policy alternatives.

The claim is deliberately narrower than “a new cloud optimizer beats existing tools.” We claim that the project operationalizes a cloud cost-optimization pipeline and empirically quantifies when conservative P95-based right-sizing and reservation heuristics diverge from capacity-reservation theory under intermittent demand surges. In particular, a $q_{0.95} \times 1.3$ rule is the conservative capacity rule used by the implemented pipeline, but can forfeit reserved-instance discounts when naively reused as a reservation-sizing fractile.

The contributions are:

- A reproducible operationalization of a cloud-cost pipeline that provides system context for studying practical right-sizing and reservation decisions.
- An empirical bridge between capacity-reservation theory and FinOps practice on a synthetic AWS account and public VM traces, including a service-level synthetic

decomposition that separates EC2 upsizing from RDS-driven aggregate savings.

- An external validation study showing that the same MILP right-sizing rule saves on restored Bitbrains and Materna traces, plus a separately generated Azure boundary case under max-based percentile and missing-memory semantics.
- An out-of-sample policy simulation that measures the price of conservatism and compares the deployed high-percentile heuristic with newsvendor, stochastic-programming, dual-capacity, and look-back replay reservation baselines.

II. RELATED WORK

The reservation side of this work builds on the classical newsvendor model and data-driven variants [5], [6], stochastic cloud provisioning [7], [8], and online instance-acquisition formulations [9], [10]. Chen et al. [1] are the closest theoretical reference: their model separates base and surge capacity and derives newsvendor-like capacity levels. We do not rederive that theory; we evaluate how an implemented right-sizing pipeline relates to it on repository and public traces.

Practical right-sizing systems such as Google Autopilot and Azure Resource Central/Doppler use utilization percentiles and headroom to avoid under-provisioning [2]–[4]; other work studies instance selection and cloud cost reduction [11]–[14]. Forecasting remains useful context for provisioning [15]–[18], but this paper sizes capacity from empirical distributions rather than point forecasts.

III. METHODOLOGY: PIPELINE AND POLICY EVALUATION

This section separates the implemented pipeline from the paper-side policy experiments. The pipeline supplies the evaluated right-sizing and recommendation behavior. The reservation-fractile and dual-capacity analyses are not deployed production features; they are held-out simulations used to measure the gap between the implemented P95 heuristic and capacity-reservation theory.

Fig. 1 summarizes the implemented system. Telemetry enters the SQLite database either from a synthetic demo account or from real AWS collection. The backend exposes costs, forecasts, recommendations, settings, and connection-management routes; the frontend renders the dashboard. The paper’s claims about optimization rely on the optimizer and forecasting modules, not on the dashboard presentation layer.

A. Forecasting and Anomaly Context

The forecasting module exposes a shared fit/predict interface for Naive, SeasonalNaive, ETS, Prophet, and SARIMAX models. Paper scripts evaluate them with walk-forward MAPE, MASE [19], interval coverage, and small-sample Diebold-Mariano tests [20], [21]. Forecasts support monitoring context; sizing uses observed utilization percentiles, which matters because real-trace aggregate forecasts are weak.

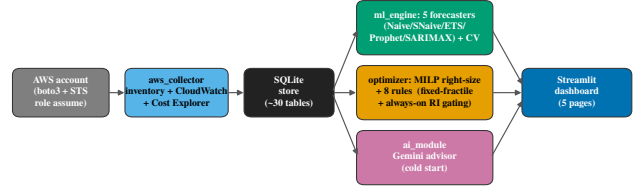


Fig. 1. Smart Cloud Optimizer architecture. The system loads telemetry into SQLite and runs forecasting, MILP right-sizing, transparent rules, and an LLM advisor; only right-sizing and reservation behavior are evaluated here.

B. MILP Right-Sizing

For compute resources, the optimizer solves a binary assignment problem. Let $\mathcal{I}$ be resources and $\mathcal{C}$ candidate instance types with price p_c , vCPUs v_c , and memory m_c . The decision variable $x_{i,c} \in \{0, 1\}$ assigns resource i to type c :

$$\min_x \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}} p_c x_{i,c} \quad (1)$$

$$\text{s.t.} \quad \sum_c x_{i,c} = 1 \quad \forall i, \quad (2)$$

$$\sum_c v_c x_{i,c} \geq r_i^{cpu} \quad \forall i, \quad (3)$$

$$\sum_c m_c x_{i,c} \geq r_i^{mem} \quad \forall i, \quad (4)$$

$$\sum_{i,c} p_c x_{i,c} \leq B \quad \text{if a cap is supplied.} \quad (5)$$

The requirement r_i uses the deployed rule:

$$r_i = q_{0.95}(u_i) V_i h, \quad h = 1.3, \quad (6)$$

where u_i is observed utilization and V_i is current capacity. EC2 uses CPU and memory when memory utilization is available; RDS uses CPU because the stored RDS memory metric is not directly comparable as a utilization percentage.

C. Rule-Based Recommendations

The rule engine adds transparent service-specific actions such as EC2/RDS reserved-pricing switches and waste-removal checks, deduplicating by resource and action type. These rules provide implementation context; the paper’s quantitative claims come from the MILP, generated evidence tables, and reservation-policy simulations.

IV. RESERVATION-FRACTILE INTERPRETATION

The system’s high-percentile rule is conservative by design. It aims to avoid performance-impacting under-provisioning when choosing instance sizes. For reservation sizing, however, the cost model is different. If a reserved unit costs w_T per period and on-demand costs w_0 , define $\gamma = w_T/w_0$. A static reserved capacity K charged against demand D has normalized expected cost

$$C(K) = \gamma K + \mathbb{E}(D - K)^+. \quad (7)$$

The newsvendor optimum chooses the empirical $(1-\gamma)$ quantile. Thus, at $\gamma \approx 0.63$, the optimal reservation service level is far below 0.95. This does not invalidate P95 right-sizing. It means that a P95 capacity guardrail and a reservation commitment level solve different problems.

The policy simulation in Section VI-C makes this distinction quantitative. It reads the deployed $q_{0.95} \times 1.3$ rule counterfactually as if it were a reservation-sizing rule, then compares it with the newsvendor fractile, a sample-average-approximation stochastic program, dual-capacity base/surge policies, look-back replay policies, and a clairvoyant lower bound.

V. EXPERIMENTAL DESIGN

A. Research Questions

The evaluation is organized around four questions:

- RQ1: What savings does the implemented optimizer report on the repository’s synthetic AWS account?
- RQ2: Does the same MILP right-sizing rule transfer to public real VM traces under a consistent price catalog?
- RQ3: How reliable are the forecasting models as a basis for capacity planning?
- RQ4: What is the out-of-sample cost of applying a conservative right-sizing fractile to reservation sizing?

B. Datasets and Inputs

Table I summarizes the datasets used by the paper scripts. Study I uses the committed synthetic AWS account. Study II uses public Bitbrains and Materna traces [22], [23], priced against a verified AWS catalog snapshot. The Azure arm uses the Azure Public Dataset V2 lineage associated with Resource Central [3].

C. Metrics

Optimization results are reported as estimated monthly cost reduction relative to the relevant baseline. Forecasting uses MAPE, MASE, empirical interval coverage, and paired statistical tests where enough aligned forecast errors exist. Policy-simulation results are normalized as realized test cost as a percentage of an on-demand-only baseline.

D. Generated Baselines and Ablations

Tables III–VI are generated by `paper/evidence_experiments.py`. The baseline and ablation rows use the committed synthetic account, but EC2 current inventory and EC2 candidates are repriced to the same snapshot AWS catalog so current and recommended costs are comparable. RDS remains on the committed demo DB price basis because the paper snapshot covers EC2 only. EC2 right-sizing costs are evaluated on a common snapshot on-demand price basis so that capacity choices are comparable, including two `m5.xlarge` current-inventory rows originally recorded at RI prices; reserved-instance commitment effects are evaluated separately in the paper-side usage-hour RI replay surrogate and in Study III. This normalization does not mean the originally reserved EC2 inventory would actually be billed at on-demand rates, and it is not a claim

about AWS Cost Explorer, Savings Plans, or commercial recommender output. If a service-level MILP is infeasible, the table marks the row as partial or failed and does not report a comparable savings percentage. The restored GWA raw-trace rerun adds Study II bootstrap intervals and a percentile baseline grid in `results_external.json` and `table_external_baselines.tex`; the Azure raw `vmtable.csv.gz` remains absent. Table VII reports a labeled usage-hour reservation replay surrogate; it is not a commercial-tool output.

E. Candidate Catalog Completeness

The synthetic-account ablations expose a feasibility boundary in the committed demo catalog. The evidence script records 13 EC2 price rows in the database, but only four EC2 rows have the positive price and hardware specifications needed by the implemented MILP; RDS has six price rows and four solver-valid rows. Under $q_{0.95} \times 1.3$ and related high-percentile or high-headroom settings, the EC2 constraints exceed that reduced candidate set, so CBC returns an infeasible EC2 assignment while the RDS subproblem remains feasible. This is a catalog-completeness and constraint-feasibility boundary, not a claim that conservative P95 sizing is universally infeasible.

To separate catalog coverage from policy infeasibility, `paper/catalog_comparison.py` reruns the same baseline and ablation grid while repricing EC2 current inventory and EC2 candidates with the documented, snapshot-dated 28-type AWS Price List catalog used by the external-validation scripts. RDS candidates remain unchanged. Table II shows that the non-budget EC2 infeasibilities disappear under this richer EC2 candidate set. The remaining EC2 infeasibilities are confined to explicit budget-cap rows, where the conservative snapshot-priced EC2 assignment exceeds the EC2 service-level cap. The paper can safely conclude that production right-sizing pipelines must validate catalog coverage and budget feasibility before applying aggressive headroom policies. It still cannot conclude that a full all-SKU AWS catalog, AWS Compute Optimizer, Azure Advisor, or a commercial FinOps tool would behave the same way.

VI. RESULTS

A. Study I: Synthetic AWS Account

The synthetic account spans 365 days from 2024-07-01 to 2025-06-30. The pricing audit in `paper/pricing_audit.md` shows that the stored EC2 current-inventory costs are mostly consistent with the snapshot catalog, but several committed `instance_pricing` candidate rows are wrong or incomplete. Consequently, the paper does not use the stale stored `recommendations` table as a headline savings result. The recomputed implementation sanity check emits 15 deduplicated recommendations worth \$507.83/mo under the committed demo DB path; this is kept as system-context evidence, not as the paper’s primary price-comparable right-sizing result.

On the corrected compute-scope price basis, current synthetic compute cost is \$1241.58/mo. The deployed $q_{0.95} \times 1.3$ MILP

TABLE I
EVALUATION INPUTS. GWA CHECKSUMS AND PROVENANCE ARE RECORDED IN `TRACE_PROVENANCE.MD`; THE AZURE RAW-INPUT CHECKSUM REMAINS UNAVAILABLE.

Input	Scale and period	Used for	Evidence file
Synthetic AWS account	365 days (2024-07-01–2025-06-30), 42 resources	optimizer savings, forecasting, policy simulation	<code>results_baselines.json</code> ; <code>results_ablations.json</code> ; <code>numbers_evidence.tex</code> ; <code>table_service_decomposition.tex</code> ; <code>results_catalog_comparison.json</code>
Bitbrains fastStorage	1250 VMs, 2013 trace	real-trace MLP right-sizing	<code>results_external.json</code> ; <code>trace_provenance.md</code>
Bitbrains Rnd	900 VMs, 2142 hourly aggregate points	right-sizing, forecasting, policy simulation	<code>trace_rnd_hourly.csv</code> ; <code>trace_provenance.md</code>
Materna-Trace-3	547 VMs, 2016 trace	second-provider right-sizing validation	<code>results_external.json</code> ; <code>trace_provenance.md</code>
Azure Public Dataset V2	237,272 filtered VMs from 2,695,548 rows, 2019 trace	boundary-condition right-sizing study	<code>results_azure.json</code> ; raw input not restored

TABLE II
CATALOG-COMPARISON RERUN FOR THE SYNTHETIC-ACCOUNT EC2 FEASIBILITY BOUNDARY. THE SNAPSHOT ROW USES A SHARED EC2 PRICE BASIS: CURRENT EC2 INVENTORY AND EC2 CANDIDATES ARE REPRICED WITH THE DOCUMENTED AWS PRICE LIST SNAPSHOT IN `AWS_CATALOG.PY`; RDS CANDIDATES REMAIN FROM THE COMMITTED DEMO DATABASE. THE 19.0% SNAPSHOT-ROW SAVING IS THE SERVICE-MIXED AGGREGATE DECOMPOSED IN TABLE IV. NON-CAP EC2 INFEASIBLE ROWS EXCLUDE EXPLICIT BUDGET-CAP CONSTRAINTS. PARTIAL ROWS ARE NOT COMPARABLE SAVINGS ESTIMATES.

Catalog/basis	EC2 raw	EC2 valid	P95×1.3 status	Cost (\$/mo)	Saving (%)	Non-cap EC2 infeas. rows	Cap EC2 infeas. rows	Boundary
Original demo DB	13	4	Partial (EC2 Infeasible)	–	–	9/18	4/4	catalog coverage
AWS snapshot shared EC2 basis	28	28	Optimal	1,006	19.0	0/18	4/4	service budget cap

EC2 infeasibility disappears for non-budget baseline and ablation rows with the shared-basis snapshot EC2 catalog; remaining EC2 infeasibility is confined to explicit per-service budget-cap rows.

TABLE III
GENERATED BASELINE COMPARISON. SYNTHETIC COMPUTE ROWS USE A SHARED PRICE BASIS: EC2 CURRENT INVENTORY AND EC2 CANDIDATES ARE REPRICED WITH THE SNAPSHOT-DATED AWS CATALOG IN `PAPER/AWS_CATALOG.PY`; RDS REMAINS ON THE COMMITTED DEMO DB BASIS BECAUSE THE PAPER SNAPSHOT COVERS EC2 ONLY. AGGREGATE SYNTHETIC SAVINGS, INCLUDING THE 19.0% P95 x 1.3 ROW, SHOULD BE READ WITH THE EC2/RDS SPLIT IN TABLE IV. PARTIAL OR FAILED ROWS ARE NOT COMPARABLE SAVINGS ESTIMATES AND ARE SHOWN AS “–”. THE REAL-TRACE ROW USES THE RESTORED GWA STUDY II LIFT-AND-SHIFT AGGREGATE; THE FULL GWA BASELINE GRID IS IN `TABLE_EXTERNAL_BASELINES.TEX`. BOOTSTRAP INTERVALS ARE DESCRIPTIVE RESOURCE-LEVEL 95% INTERVALS.

Method	Resources	Cost (\$/mo)	Saving (%)	95% CI (%)	Down	Up	Status
Current inventory	10	1,242	0.0	[0.0, 0.0]	0	0	Baseline
Mean utilization	10	347	72.0	[48.0, 85.5]	9	0	Optimal
Median utilization	10	332	73.3	[50.5, 86.4]	9	0	Optimal
P90 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
P95 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
P95 x 1.3 heuristic	10	1,006	19.0	[-48.0, 72.5]	8	2	Optimal
P99 sizing	10	660	46.8	[17.5, 74.9]	9	0	Optimal
Max utilization	10	736	40.7	[10.3, 69.2]	7	0	Optimal
Lift-and-shift to P95 x 1.3	2297	126,426	41.8	–	1575	242	existing aggregate; raw per-VM CI not rerun

TABLE IV
STUDY I SERVICE-LEVEL DECOMPOSITION FOR THE GENERATED $q_{0.95} \times 1.3$ ROW. EC2 CURRENT INVENTORY AND CANDIDATES USE THE SHARED SNAPSHOT ON-DEMAND BASIS; RDS REMAINS ON THE COMMITTED DEMO DB BASIS. POSITIVE DELTAS INDICATE COST INCREASES.

Scope	Current (\$/mo)	Optimized (\$/mo)	Delta (\$/mo)	Delta (%)
EC2	844	971	+127	+15.0
RDS	398	35	-363	-91.2
Aggregate	1,242	1,006	-236	-19.0

row is feasible and costs \$1005.65/mo. Table IV shows why the aggregate 19.0% reduction is not an unqualified optimizer win: EC2 increases from \$843.73/mo to \$970.61/mo, a \$126.87/mo (15.0%) increase, while RDS decreases from \$397.85/mo to \$35.04/mo, a \$362.81/mo (91.2%) decrease. Since RDS remains on the committed demo DB price basis, the aggregate reduction should be read as a method-illustrative service-mix

result. The EC2 increase is the more direct evidence for the paper’s boundary condition: conservative headroom can upsize capacity and reduce or reverse service-level savings.

The usage-hour replay surrogate recommends 6 EC2 reservations over the 7/30/60 day look-back windows, saving \$209.15/mo (37.1%) on eligible EC2 on-demand spend. The rows collapse because all six eligible synthetic on-demand EC2 instances have full metric coverage in every window and the snapshot reserved/on-demand prices are static. These are synthetic, method-illustrative estimates and not realized production savings.

Forecasting is useful but not decisive for the cost results. Table VIII shows walk-forward MAPE on the synthetic account. ETS has the best mean at the 30-day horizon (11.2%), but the ETS versus seasonal-naive difference is not statistically significant at the reported sample size ($p = 0.198$). The paper therefore treats ETS as the selected model for monitoring, not as proof of forecasting superiority.

Tables III and V further qualify Study I. The generated

TABLE V

GENERATED ABLATION RESULTS ON THE SYNTHETIC ACCOUNT WITH EC2 REPRICED TO THE SNAPSHOT AWS CATALOG. PERCENTILE, HEADROOM, MEMORY, AND BUDGET ROWS REPORT ASSIGNMENT COST ONLY FOR FEASIBLE MILP SOLVES; AGGREGATE 19.0% ROWS INHERIT THE EC2/RDS SPLIT AND RDS DEMO-PRICE CAVEAT IN TABLE IV. PARTIAL OR FAILED ROWS EXPOSE FEASIBILITY LIMITS AND ARE NOT COMPARABLE SAVINGS ESTIMATES. RULE-SET ROWS ARE A RECOMPUTED IMPLEMENTATION SANITY CHECK ON THE COMMITTED DEMO DB, NOT THE SOURCE OF HEADLINE PAPER SAVINGS.

Dimension	Level	Cost (\$/mo)	Saving (%)	CPU exceed (%)	Recs/vars	Status
percentile q	0.75	478	61.5	1.2	232	Optimal
percentile q	0.90	1,006	19.0	0.0	232	Optimal
percentile q	0.95	1,006	19.0	0.0	232	Optimal
percentile q	0.99	1,141	8.1	0.0	232	Optimal
headroom h	1.0	660	46.8	0.0	232	Optimal
headroom h	1.1	660	46.8	0.0	232	Optimal
headroom h	1.3	1,006	19.0	0.0	232	Optimal
headroom h	1.5	1,141	8.1	0.0	232	Optimal
memory constraint	off	585	52.9	0.0	232	Optimal
memory constraint	observed	1,006	19.0	0.0	232	Optimal
budget cap	none	1,006	19.0	0.0	232	Optimal
budget cap	100% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	90% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	75% current	—	—	—	232	Partial (EC2 Infeasible)
budget cap	1% current	—	—	—	232	Failed (EC2 Infeasible, RDS Infeasible)
rule set	MILP right-sizing only	—	—	—	2	\$362.81/mo
rule set	reserved pricing only	—	—	—	2	\$125.85/mo
rule set	waste rules only	—	—	—	11	\$19.17/mo
rule set	all rules, deduplicated	—	—	—	15	\$507.83/mo

TABLE VI

RUNTIME AND CLONE-STRESS DIAGNOSTICS FOR THE GENERATED EVIDENCE SCRIPT. GWA BITBRAINS/MATERNAL RAW-TRACE PARSING RUNTIMES ARE RECORDED IN `TRACE_PROVENANCE.MD`; AZURE RAW PARSING RUNTIME WAS NOT RECORDED BECAUSE THE RAW `VMTABLE.CSV.GZ` INPUT WAS NOT RESTORED. THESE DIAGNOSTICS ARE NOT A HARDWARE-NORMALIZED SCALABILITY STUDY.

Experiment	<i>n</i>	Vars	Solves	Solve s	Peak MB	Status
baseline sweep	10	1624	14	0.117	114.1	computed
ablation sweep	10	3480	30	0.248	118.6	computed
rule-set ablation	10	0	0	0.557	118.6	computed
EC2 MILP stress <i>n</i> =100	100	2800	1	0.071	120.4	Optimal
EC2 MILP stress <i>n</i> =500	500	14000	1	0.396	141.5	Optimal
EC2 MILP stress <i>n</i> =1000	1000	28000	1	1.285	171.4	Optimal
evidence script total	10	49904	47	7.860	171.4	computed

TABLE VII

PAPER-SIDE EC2 USAGE-HOUR RI REPLAY BASELINE. THE REPLAY USES TRAILING OBSERVED METRIC COVERAGE AND THE SAME SNAPSHOT EC2 ON-DEMAND/RESERVED PRICES; IT IS NOT AWS COMPUTE OPTIMIZER, COST EXPLORER, AZURE ADVISOR, OR THIRD-PARTY RECOMMENDER OUTPUT. THE 7-, 30-, AND 60-DAY WINDOWS COLLAPSE TO ONE ROW BECAUSE THEY SELECT THE SAME SIX FULLY OBSERVED SYNTHETIC EC2 INSTANCES.

Lookbacks (days)	Recs	Current (\$/mo)	Saving (\$/mo)	Saving (%)	Status
7/30/60	6	563	209	37.1	paper-side replay

baseline table uses the snapshot EC2 current/candidate basis, where the non-budget $q_{0.95} \times 1.3$ row is feasible. The catalog comparison in Table II preserves the original demo catalog

TABLE VIII

STUDY I SYNTHETIC-ACCOUNT WALK-FORWARD MAPE (%), MEAN $\pm$ SD). THE SOURCE TABLE IS GENERATED BY `MAKE_FIGURES.PY`.

Model	<i>h</i> =7	<i>h</i> =14	<i>h</i> =30	<i>h</i> =60	<i>h</i> =90
ETS (selected)	11.28 $\pm$ 2.54	11.24 $\pm$ 2.45	11.20 $\pm$ 0.57	11.39 $\pm$ 0.74	12.15 $\pm$ 2.39
SeasonalNaive	10.18 $\pm$ 2.84	10.84 $\pm$ 2.75	12.28 $\pm$ 2.82	12.66 $\pm$ 2.21	12.91 $\pm$ 2.02
Prophet	14.41 $\pm$ 6.22	20.47 $\pm$ 7.40	20.93 $\pm$ 7.53	19.70 $\pm$ 7.89	18.08 $\pm$ 8.42
Naive	25.16 $\pm$ 4.47	27.90 $\pm$ 3.52	27.26 $\pm$ 4.36	27.87 $\pm$ 4.09	27.38 $\pm$ 3.70

outcome: with only four solver-valid EC2 candidates, the EC2 subproblem is infeasible under the same policy while RDS remains feasible. Partial original-demo rows are therefore marked as non-comparable savings estimates; fallback costs are retained only in JSON diagnostics. These rows should be read as a feasibility boundary condition, not as evidence of a complete EC2 right-sizing solution or as a universal failure of P95 right-sizing. Replacing the incomplete demo EC2 catalog with the documented 28-type snapshot EC2 candidate set makes the non-budget P95 and high-percentile/headroom rows feasible, while explicit per-service budget caps remain a separate infeasibility boundary.

B. Study II: External Real-Trace Validation

Table IX reports real-trace right-sizing from restored GWA raw traces. The baseline maps each VM to the cheapest AWS type covering provisioned cores and memory. The optimized assignment uses the deployed $q_{0.95} \times 1.3$ requirements. On Bitbrains fastStorage and Rnd, right-sizing saves 33.6% (95% CI 24.5–41.8%) and 33.1% (19.3–46.3%), respectively. On

TABLE IX
STUDY II RESTORED-GWA RIGHT-SIZING AT AWS ON-DEMAND RATES
FROM SNAPSHOT 2026-06-30. THE +RI COLUMN APPLIES ONE-YEAR
RESERVED PRICING.

Dataset	VMs	Baseline (\$/mo)	Optimized (\$/mo)	Saving / +RI (%)
fastStorage	1250	128,588	85,414	33.6 / 58.3
Rnd	500	44,294	29,614	33.1 / 58.0
Materna	547	44,184	11,398	74.2 / 83.8

TABLE X
STUDY II GWA BASELINE GRID: SAVINGS VS. LIFT-AND-SHIFT (%). FULL
CIS AND COSTS ARE IN TABLE_EXTERNAL_BASELINES.TEX.

Policy	fastStorage	Rnd	Materna
Lift-and-shift	0.0	0.0	0.0
Mean	81.4	82.6	86.1
Median	86.8	90.4	86.5
P90	46.6	42.0	81.4
P95	40.2	37.4	78.8
P99	34.0	32.1	73.3
Max	7.3	-3.7	42.7
P95 x 1.3	33.6	33.1	74.2

Materna, the saving is 74.2% (71.4–76.8%), reflecting much larger over-provisioning in that trace. Across all restored GWA VMs, the aggregate saving is 41.8% (35.9–47.5%). Applying one-year reserved pricing to the optimized always-on trace fleet raises total reductions to 58.3%, 58.0%, and 83.8%. Using one-year RI pricing, the pooled mapped reduction is 63.4% (95% CI 59.7–67.0%).

Table X shows that mean, median, and less-conservative percentile sizing can produce higher apparent mapped savings than $q_{0.95} \times 1.3$. This does not make them better policies by itself: the paper does not yet evaluate SLO or performance-risk violations after right-sizing. The result instead exposes the savings-risk frontier central to the paper: conservative headroom reduces risk exposure but can sacrifice savings, so P95 x 1.3 should not be oversold as savings-optimal.

The Azure study is deliberately included as a negative boundary condition, but it was not rerun in this workspace because the raw V2 `vmtable.csv.gz` input is absent. The committed generated result retains 237,272 filtered VMs. Since Azure publishes a max-based CPU percentile and no per-VM memory-usage series in the `vmtable` input, memory is held at the provisioned bucket. Under those semantics, right-sizing alone is cost-negative (-3.8%), although reserved pricing still yields 34.8%. This result limits the generality of P95 right-sizing: the rule depends on what the utilization percentile means and on which resources are measurable.

Forecasting the real aggregate is difficult: at 24 hours the best hourly model is Prophet with 60.8% MAPE, and tested pairwise differences are not statistically significant (e.g., ETS vs. seasonal-naive $p = 0.232$). Table XI supports treating forecasts as monitoring context rather than savings drivers.

TABLE XI
STUDY II REAL RND AGGREGATE FORECASTING MAPE (%), MEAN $\pm$ SD).

Model	$h=6$ h	$h=12$ h	$h=24$ h	$h=48$ h	$h=7$ d	$h=14$ d
ETS	48.2 $\pm$ 46.2	71.9 $\pm$ 55.4	77.0 $\pm$ 57.2	91.2 $\pm$ 47.8	52.1 $\pm$ 29.9	64.4 $\pm$ 40.9
SeasonalNaive	65.9 $\pm$ 33.6	73.7 $\pm$ 46.3	65.0 $\pm$ 40.3	65.8 $\pm$ 33.8	54.2 $\pm$ 22.6	59.7 $\pm$ 32.9
Prophet	49.0 $\pm$ 26.2	60.5 $\pm$ 30.0	60.8 $\pm$ 21.6	88.1 $\pm$ 42.3	59.8 $\pm$ 36.9	66.7 $\pm$ 30.0
Naive	57.4 $\pm$ 71.2	87.9 $\pm$ 86.1	82.3 $\pm$ 66.6	94.2 $\pm$ 55.7	53.4 $\pm$ 14.2	66.5 $\pm$ 44.5

C. Study III: Price of Conservatism

The policy simulation splits demand chronologically into train and test segments, sizes each policy on training data, and charges it on held-out test demand. The synthetic run uses 219 training days and 146 test days; the real-trace run uses 1285 training hours and 857 test hours. The theoretical formulation expresses demand and commitment in physical-capacity units. In the empirical evaluation, the synthetic arm reads daily account cost, so its input series and commitment proxy are measured in USD/day, whereas the restored Rnd arm is measured in MHz. The synthetic arm therefore evaluates policy behavior on a cost-proxy time series rather than simulating physical-capacity allocation or provider billing. At the catalog reserved/on-demand ratio $\gamma = 0.63$, the deployed $q_{0.95} \times 1.3$ heuristic, read counterfactually as a reservation-sizing rule, realizes 101.8% of on-demand cost on the synthetic account and 153.5% on the real trace. The newsvendor fractile realizes 73.6% and 86.2%. To avoid comparing only against a deliberately conservative strawman, Tables XII and XIII also include paper-side 7-, 30-, and 60-day look-back replay policies. These results are paper-side replay surrogates and are not outputs from AWS, Azure, or another commercial recommendation system. On the real trace at $\gamma = 0.63$, the 30-day replay realizes 84.9% of on-demand cost, while the 7-day replay realizes 91.2%, showing that look-back choice matters under bursts. The real-trace 60-day row equals newsvendor because the training split is shorter than the requested 60-day window, so the replay uses the full training set.

Fig. 2 and Fig. 3 plot the cost curves. Tables XII and XIII give selected discount levels. The SAA stochastic program recovers the newsvendor quantile closely, which validates the closed-form policy and the simulator rather than establishing a separate competing method. The dual-capacity policy adds little on the synthetic daily series but materially improves the real trace, where surge states persist long enough for causal activation to retain most of the value. The look-back rows are therefore a sanity-check replay family, not a substitute for a direct commercial recommender baseline.

VII. DISCUSSION

A. What the System Can Safely Claim

The repository supports three defensible claims. First, the implemented optimizer combines MILP right-sizing and transparent service rules. Second, the paper scripts reproduce a multi-part evaluation from database, public traces, and a snapshot-dated price catalog. Third, the policy simulation shows

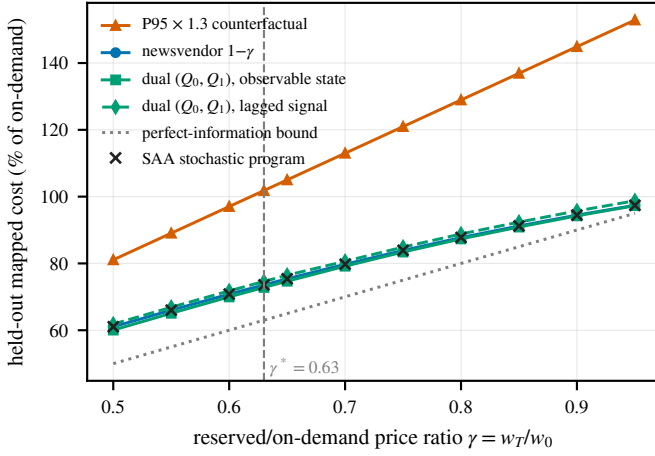


Fig. 2. Study III synthetic policy simulation: realized held-out cost as a percentage of on-demand-only cost across reserved/on-demand ratios.

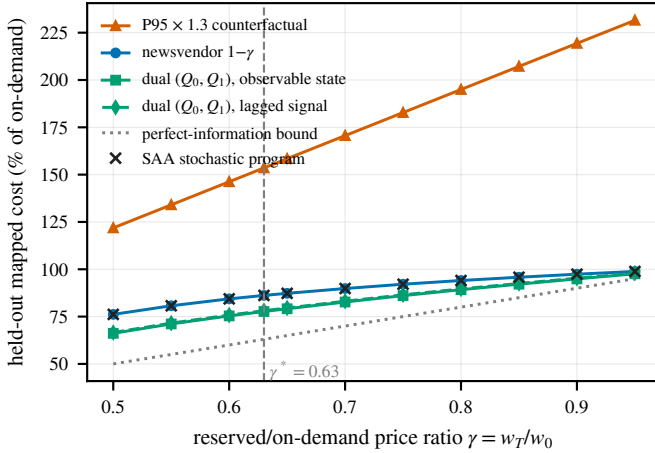


Fig. 3. Study III real-trace policy simulation on Bitbrains Rnd aggregate demand. The dual-capacity variants are most valuable when surge states persist.

that a high right-sizing percentile should not be treated as a reservation sizing rule without accounting for γ .

B. What Should Not Be Oversold

The deployed system does not implement the full dynamic (s, S) policy of Chen et al. [1]; it implements static high-percentile right-sizing and always-on reserved-pricing checks. The LLM advisor is not evaluated for savings. The external baselines are internal mathematical and replay baselines, not direct runs of commercial recommenders. Therefore, the paper should not claim commercial superiority. The synthetic EC2 $q_{0.95} \times 1.3$ infeasibility under the committed demo catalog also means the paper should not present that original EC2 result as a successful full-fleet EC2 right-sizing solve. The snapshot-catalog rerun supports the catalog-coverage interpretation, but it is not a commercial-tool or all-SKU AWS comparison.

TABLE XII
STUDY III SYNTHETIC POLICY COSTS AS % OF ON-DEMAND-ONLY.

Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
P95 $\times$ 1.3 counterfactual	97.0	113.0	128.9	144.9
newsvendor $1-\gamma$	70.8	79.7	87.7	94.5
SAA stochastic program	70.8	79.8	87.8	94.5
7-day recency-window replay	71.0	80.2	87.6	94.5
30-day recency-window replay	70.5	79.7	87.4	94.3
60-day recency-window replay	70.5	79.8	87.5	94.6
dual (Q_0, Q_1) , obs. state	70.0	79.2	87.3	94.2
dual (Q_0, Q_1) , lagged signal	71.8	80.6	88.8	95.7
perfect-information bound	60.0	70.0	80.0	90.0

TABLE XIII
STUDY III REAL-TRACE POLICY COSTS AS % OF ON-DEMAND-ONLY.

Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
P95 $\times$ 1.3 counterfactual	146.2	170.6	195.0	219.3
newsvendor $1-\gamma$	84.4	89.8	94.1	97.4
SAA stochastic program	84.4	89.8	94.1	97.4
7-day recency-window replay	89.3	95.6	102.4	103.6
30-day recency-window replay	82.7	89.2	93.8	97.3
60-day recency-window replay	84.4	89.8	94.1	97.4
dual (Q_0, Q_1) , obs. state	75.4	82.7	89.2	94.9
dual (Q_0, Q_1) , lagged signal	75.8	83.1	89.6	95.3
perfect-information bound	60.0	70.0	80.0	90.0

C. Artifact Scope and Remaining Extensions

The minimum synthetic-account baselines, ablations, service decomposition, and runtime measurements are now generated in Tables III–VI. The catalog comparison in Table II also reduces the synthetic EC2 feasibility risk by showing that non-budget infeasibility disappears under the shared-basis snapshot EC2 candidate set. The paper-side usage-hour RI replay surrogate in Table VII partially addresses the missing-baseline risk but does not replace a direct commercial tool comparison. The restored GWA Study II rerun adds checksums, bootstrap intervals, and a baseline grid for Bitbrains and Materna; remaining extensions include direct commercial baselines, Azure raw-vmtable restoration with timing and intervals, SLO/performance-risk validation, all-SKU/region catalog sensitivity, and a full dynamic (s, S) reservation implementation.

VIII. THREATS TO VALIDITY

Synthetic data and demo pricing. Study I uses the repository’s synthetic AWS account. Its dollar savings are method-illustrative and should not be presented as production savings. The committed demo EC2 candidate price table contains incorrect or incomplete rows, so the paper separates original demo-price diagnostics from snapshot-repriced EC2 results. The aggregate Study I compute reduction is dominated by RDS, which remains demo-priced; it should not be used as a production-dollar or cross-service superiority claim.

Trace age and mapping. Bitbrains and Materna are older managed-hosting traces, and their VMs are priced using a 2026 AWS catalog. The GWA inputs were restored with checksums

for this rerun, but percentages are useful for method comparison under the mapping, not as absolute production-dollar evidence.

Metric semantics. The Azure result shows that the meaning of a published percentile matters. Max-based percentiles and unavailable memory usage can reverse the right-sizing result.

Candidate catalog completeness. The committed synthetic DB contains more EC2 price rows than solver-valid EC2 hardware rows. Consequently, high percentile and high-headroom EC2 ablations can be infeasible because the reduced candidate set cannot satisfy the induced CPU/memory requirements. This limits synthetic-account conclusions about EC2 right-sizing. The paper-local catalog comparison reruns the synthetic ablations with a richer 28-type, snapshot-dated EC2 candidate set and a shared EC2 current/candidate price basis, and keeps infeasible rows visible. That rerun removes non-budget EC2 infeasibility, but it is still not an all-SKU AWS catalog study and does not replace a direct commercial recommender baseline.

Policy abstraction. Study III uses aggregate demand and simplified reservation costs. It does not model all real contract terms, partial upfront payments, cancellation, resale, instance families, or capacity fragmentation.

Baselines. The SAA baseline is a correctness/replay baseline and closely matches newsvendor theory by construction. The usage-hour reservation replay and Study III look-back replay rows are paper-side replay surrogates only. The paper currently lacks a direct commercial-tool comparison.

Statistical power. Forecasting tests use limited walk-forward folds. The reported non-significant differences should remain non-claims rather than being reframed as superiority.

IX. CONCLUSION

This paper is best framed as an empirical theory-practice bridge, not as a product demonstration. Smart Cloud Optimizer operationalizes a FinOps-style pipeline with forecasting, $q_{0.95} \times 1.3$ MILP right-sizing, reserved pricing checks, and service-specific waste rules. On the repository's synthetic account, the paper no longer treats the stale stored recommendation table as a headline savings source; after EC2 snapshot repricing, the $q_{0.95} \times 1.3$ row is feasible but service-mixed: EC2 cost increases by 15.0%, while demo-priced RDS cost decreases by 91.2%, producing an aggregate compute-scope reduction of 19.0%. The aggregate should therefore be interpreted as a boundary-condition result about conservative headroom and service mix, not as an unqualified optimizer win. The paper-side reservation replay surrogate saves \$209.15/mo on eligible EC2 on-demand spend in a collapsed 7/30/60-day synthetic sanity check. On restored public Bitbrains and Materna VM traces, the same MILP right-sizing rule saves 41.8% in aggregate against a lift-and-shift baseline (95% CI 35.9–47.5%), while the Azure boundary case, whose raw input was not restored, shows that metric semantics can make the heuristic fail. The policy simulation then isolates the central research insight: under intermittent demand surges, conservative P95-based right-sizing and price-aware capacity-reservation theory can diverge sharply. At the observed reserved/on-demand ratio, using the high-percentile capacity rule as a reservation-sizing rule forfeits

the discount, whereas the newsvendor and look-back replay policies retain much more of it. Remaining extensions include direct commercial baselines, Azure raw-vmtable restoration with confidence intervals and runtime measurements, broader all-SKU/region catalog sensitivity, SLO validation, and live or modern production data.

AUTHOR CONTACT INFORMATION

Corresponding author: Ibrahim Mohamed: ibrahimmohamedabdelsadek@gmail.com.

Author emails: Mariam Emad: mariamemadcs@gmail.com; Hazem Ibrahim: Ihazem28@gmail.com; Ahmed Sameh: ahmed.sameh.12543@gmail.com; Mahmoud Ahmed: mahmoudkame19102003@gmail.com; John Ehab: ehajohn22@gmail.com.

REFERENCES

- [1] S. Chen, J. Lei, and K. Moinsadeh, "Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges," *Production and Operations Management*, vol. 33, no. 6, pp. 1265–1284, 2024, also available as SSRN working paper 3784812.
- [2] K. Rzadca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, "Autopilot: Workload autoscaling at Google," in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*. ACM, 2020, pp. 1–16.
- [3] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 153–167.
- [4] J. Cahoon, W. Wang, Y. Zhu, K. Lin, S. Liu, R. Truong, N. Singh, C. Wan, A. Ciorcia, S. Narasimhan, and S. Krishnan, "Doppler: Automated SKU recommendation in migrating SQL workloads to the cloud," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3509–3521, 2022.
- [5] K. J. Arrow, T. Harris, and J. Marschak, "Optimal inventory policy," *Econometrica*, vol. 19, no. 3, pp. 250–272, 1951.
- [6] G.-Y. Ban and C. Rudin, "The big data newsvendor: Practical insights from machine learning," *Operations Research*, vol. 67, no. 1, pp. 90–108, 2019.
- [7] S. Chaisiri, B.-S. Lee, and D. Niyato, "Optimization of resource provisioning cost in cloud computing," *IEEE Transactions on Services Computing*, vol. 5, no. 2, pp. 164–177, 2012.
- [8] K. Bülbül, N. Noyan, and H. Erol, "Multi-stage stochastic programming models for provisioning cloud computing resources," *European Journal of Operational Research*, vol. 288, no. 3, pp. 886–901, 2021.
- [9] W. Wang, B. Liang, and B. Li, "Optimal online multi-instance acquisition in IaaS clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 12, pp. 3407–3419, 2015.
- [10] W. Wang, D. Niu, B. Liang, and B. Li, "Dynamic cloud instance acquisition via IaaS cloud brokerage," *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 6, pp. 1580–1593, 2015.
- [11] N. J. Yadwadkar, B. Hariharan, J. E. Gonzalez, B. Smith, and R. H. Katz, "Selecting the best VM across multiple public clouds: A data-driven performance modeling approach," in *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*. New York, NY, USA: ACM, 2017, pp. 452–465.
- [12] N. Aydın, İ. Muter, and Ş. İ. Birbil, "Multi-objective temporal bin packing problem: An application in cloud computing," *Computers & Operations Research*, vol. 121, p. 104959, 2020.
- [13] M. Alves and F. Morais, "IaaS capacity planning with multiple pricing models using predictive heuristics," in *2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. Shenzhen, China: IEEE, 2025, pp. 1–8.
- [14] S. Musa and D. G. Down, "Cost-effective cloud resource provisioning using linear regression," in *2024 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. Abu Dhabi, United Arab Emirates: IEEE, 2024, pp. 176–183.

- [15] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [16] R. J. Hyndman and Y. Khandakar, "Automatic time series forecasting: The forecast package for R," *Journal of Statistical Software*, vol. 27, no. 3, pp. 1–22, 2008.
- [17] D. a. F. Kirchoff, V. Meyer, R. N. Calheiros, and C. A. F. De Rose, "Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for cloud environments," *The Journal of Supercomputing*, vol. 80, no. 15, pp. 21 920–21 951, 2024.
- [18] F. Agulló, A. Gutierrez-Torre, J. Torres, and J. L. Berral, "Enhancing the output of time series forecasting algorithms for cloud resource provisioning," *Future Generation Computer Systems*, vol. 170, p. 107833, 2025.
- [19] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.
- [20] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995.
- [21] D. Harvey, S. Leybourne, and P. Newbold, "Testing the equality of prediction mean squared errors," *International Journal of Forecasting*, vol. 13, no. 2, pp. 281–291, 1997.
- [22] S. Shen, V. van Beek, and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," in *2015 15th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474.
- [23] A. Iosup, H. Li, M. Jan, S. Anoep, C. Dumitrescu, L. Wolters, and D. H. J. Epema, "The Grid workloads archive," *Future Generation Computer Systems*, vol. 24, no. 7, pp. 672–686, 2008.